Code-Layout, Readability and Reusability

Date	July 3, 2026
Team ID	xxxxxx
Project Name	Credit Card Approval Prediction System
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP8-compliant 4-space indentation maintained across all Python files.
2	Proper File Structure	Files and folders are logically organized	Yes	Organized into templates/, static/, model/, and app.py following Flask conventions.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as income, credit_history, and loan_status clearly reflect their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions like preprocess_data(), train_model(), and predict_approval() are descriptively named.
5	Code Comments	Inline and block comments explain logic	Partial	Core preprocessing and model training steps are commented; some utility functions need more inline documentation.
6	Modular Design	Code is split into reusable functions/modules	Yes	Data preprocessing, model training, and prediction logic are separated into distinct modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Duplicate preprocessing logic was consolidated into a shared utility function.

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Form input validation is implemented and model loading/prediction is wrapped in try/except, but coverage is not yet comprehensive.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	data_preprocessing.py	Python (Pandas, NumPy)	Used in both the model training notebook and the Flask app for consistent feature encoding and cleaning.	High
2	model_training.py	Python (Scikit-learn, XGBoost)	Used to train and compare Logistic Regression, Decision Tree, Random Forest, and XGBoost models.	High
3	predict.py (model loader)	Python (Pickle, Flask)	Used in app.py to load the saved trained model and generate live predictions.	High
4	app.py (Flask routes)	Python (Flask)	Handles web form submission, input validation, and rendering of prediction results.	Medium
5	index.html / result.html	HTML, CSS, Bootstrap	Reused across the application form and prediction result pages for a consistent UI.	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear folder structure separating data preprocessing, model training, and the Flask app.
Readability	4	Descriptive variable and function names with mostly consistent formatting throughout.
Reusability	4	Preprocessing and model-training logic are reusable across the notebook and the web app.
Documentation / Comments	3	Key sections are commented; some utility functions could use more detailed inline documentation.

Aspect	Rating (1-5)	Comments
Overall Score	**4**	Well-structured, readable, and reasonably reusable codebase overall.